UniEats: The Last 500 Feet

Lynn Morris

Adviser: Robert Fish

January 8, 2026

Abstract

Food delivery apps like Uber Eats and DoorDash have revolutionized the way people order food. Despite being widely successful, these services fail at providing a complete experience to closed university campuses like Princeton University. They suffer from a problem I call the "Last 500 Feet," because their drivers are unable to get past security gates or enter locked campus buildings. This forces students to stop studying or leave their room and walk out to the street in order to meet their driver, defeating the entire purpose of ordering delivery: convenience. In this paper, I introduce UniEats, a delivery network and application designed to solve this problem. Instead of using outside drivers, UniEats uses university students who already have the necessary prox access to enter campus buildings and dorms, allowing food to be delivered directly to another student's door. A full web app was built to test the concept and a pilot study was run in November 2025. The pilot was a success, with 40 interested students and 9 completed deliveries direct to dorm with no hassles; showing that using student drivers is an effective way to avoid the security barriers that block current industry giants.

Hosted App: https://unieats-back-3813ec8fb7e1.herokuapp.com/

1. Introduction

The food delivery industry has grown massively in the last few years. Companies like Uber Eats, DoorDash, and GrubHub have drastically changed how many people eat. Together, they create a global market that is expected to generate over 200 billion USD in revenue by 2030 [1]. These services offer one simple promise: great food delivered directly to your doorstep. This model is very successful in cities and suburbs, but it faces a significant challenge when it comes to the strict security and layout of university campuses like Princeton. I call this problem the "Last 500 Feet." It is essentially the last bit of a distance that the current food delivery services cannot cover, forcing the student to make up for it themselves.

This problem exists because the current delivery model relies on open access for vehicles and drivers, neither of which exists on closed campuses. Campuses like Princeton are designed for students, not cars. Outside of a few select times of year, most vehicles are generally restricted from driving on the inner campus roads. This means that drivers cannot simply pull up to the curb of a dorm building as they would at a usual delivery location. Even if they got to the dorm, they would hit another roadblock: building security. Every residential college and building at Princeton is locked and requires "prox" (keycard) access to enter. Drivers from major delivery apps do not have this prox access, so they are unable to reach a student's door.

1.1. Motivation

These barriers make the delivery experience for a Princeton student broken. When a student orders food, the driver can only make it as far as the public street or main entryways, leaving the service unfinished. This creates a frustrating situation for the student. The reason they order delivery is because they are unable, don't have the time, or simply don't want to leave their room. They might be sick, or in the middle of exams and need every minute to study. They pay for the delivery service to save time and energy. But, instead of getting to stay in their room, the student is forced

to stop working, get dressed, and walk the "Last 500 Feet" to the edge of campus and find their driver. This removes the convenience of delivery, defeating the purpose of ordering it in the first place.

1.2. Goal and Solution

The goal of UniEats is to fix the problem of the "Last 500 Feet" by creating a system that provides delivery directly to a student's dorm room, rather than the edge of campus. UniEats solves this by changing who the drivers are. Instead of using outside drivers, UniEats uses a local network of fellow students. Since the drivers are students, they are constantly on campus and have the necessary "prox" access required to enter residential colleges and other campus buildings. They can easily get through security barriers and navigate campus to find specific rooms. This puts the convenience back in delivery, allowing students to stay in their room while their food is being delivered right to their door.

2. Background and Related Work

To understand why a system like UniEats is necessary, we need to look at the current food delivery industry.

2.1. The Commercial Food Delivery Landscape

A few giant companies currently dominate the food delivery market. When most people think of food delivery, they likely think of Uber Eats, DoorDash, or Grubhub. These companies have completely transformed the restaurant industry. Financially, these companies are massive. Industry reports estimate that the food delivery sector will generate over 200 billion USD in revenue by 2030 [1]. They have achieved this by building a highly specialized logistics network. Their model relies on "gig workers": freelance drivers who use their own personal vehicles to pick up food from restaurants and drive it to customers.

This model is designed for speed and volume. The goal for a delivery driver is to pick up an order and drop it off as quickly as possible so they can move on to the next one. This works perfectly in a standard residential neighborhood. If a customer lives in a house in the suburbs, the driver can pull into the driveway, walk ten feet to the front door, drop off the food, and leave. If the customer lives in an apartment complex, there is usually a lobby or a front desk where the food can be left.

However, they operate on a one-size-fits-all approach. They treat a delivery to a gated university campus the same way they treat a delivery to a suburban house. They assume that their drivers can reach the building address and that the customer is easily accessible. As we will see, neither of these assumptions are true at Princeton. Despite their billions in revenue, these companies do not tailor their services to students' specific needs. They view the student market as just another set of addresses, failing to account for the unique barriers that exist on a closed campus.

2.2. The "Last 500 Feet"

In logistics, the "Last Mile" typically refers to the final leg of a product's journey from a warehouse to the customer's doorstep. For Princeton students, the "Last Mile" is actually quite efficient. The driver has no trouble getting from the restaurant to the edge of campus. The failure happens in the short distance between the public street and the actual dorm room. This gap arises from a clash between the delivery driver's mode of transport and the campus design. Commercial drivers are almost exclusively in cars. Princeton's campus, however, is designed for pedestrians. Unauthorized vehicles are generally not permitted to drive on the inner pathways connecting the residential colleges. A driver cannot drive their car up to the front entrance of a building like Butler or Yeh College. They are forced to park on a public street or in a visitor lot at the edge of the campus, where the student then has to pick up their food themselves.

The result is an unsatisfactory user experience. When the driver hits this barrier, they have no choice but to stop and wait for the student. At this point, the labor shifts from the delivery app to the student. The student, who paid for delivery, has to finish the exchange by walking to meet the driver somewhere outside. They have to:

1. Stop their academic work.
2. Get dressed and prepared for the weather (which can be harsh in Princeton winters).
3. Walk a few minutes to the street where the driver is waiting.
4. Navigate the confusion of finding the correct car among many others.
5. Walk all the way back to their room.

For a sick student, this physical exertion can be harmful. For a student studying for finals, this 10-20 minute interruption breaks their focus and reduces their productivity. This doesn't even account for the fact that they must constantly check their phone to make sure they meet the driver at the

designated time. The delivery giants have solved the problem of getting food near the student, but they have entirely failed to deliver it to the student.

2.3. Campus Security

Even if a delivery driver decides to park their car and walk onto campus, they immediately encounter a second, even more difficult barrier: building security. Universities like Princeton operate where every residential college, dormitory, and academic building is locked. However, academic buildings remain open until certain times in the day. Access is controlled by an electronic system using "prox's." Students, faculty, and staff carry ID cards (the prox) that allow them to unlock these doors. This infrastructure creates a hard stop for current delivery models.

Companies like Uber Eats and DoorDash have high driver turnover. It would be a massive security risk for the university to grant building access to thousands of unaffiliated, temporary delivery drivers. As a result, these drivers are strictly treated as visitors. This creates a disconnect in the service. The service the app offers is convenience, but campus security prevents it from being delivered. When a driver arrives at a building like New College West or Yeh College, they are physically locked out. They cannot enter the lobby, use the elevator, or knock on a dorm room door.

This shows that the meet at the street scenario is not just a result of lazy drivers; it is a structural necessity of the current system. The external driver physically cannot complete the job. This leaves the "Last 500 Feet" entirely up to the student to solve.

2.4. Limitations of Current Campus Food Options

Since external delivery is broken, students are currently left with a few internal options for getting food at Princeton. However, upon closer inspection, none of these options solves the specific delivery problem.

Dining Halls and Eating Clubs

The primary source of food for students is the university dining hall system. While these provide meals, they operate on a strictly walk-in basis. A student must physically go to the location, swipe in, and eat. This works for day-to-day life, but it fails for students sick in bed or pulling an all-nighter for an exam. If a student is too ill to go outside, a dining hall is not a viable option. Eating clubs are even less convenient, often farther from student dorms, and require students to be members to use them. The dining halls do offer a service called lunch to go, but it is exclusively for lunch and still requires the student to pick up their food.

Frist Campus Center (Late Meal)

For late-night food, students often rely on "Late Meal" at the Frist Campus Center. This is a very popular option, but it suffers from the same problem: students have to go there to pick up the food. Due to the convenience of mobile ordering and simple pickup, especially at the Frist Grill, Late Meal already has many core foundations to be a valuable food delivery service for students on campus. It just needs a courier system in place.

2.5. Campus Mail Services

It is also worth comparing food delivery to package delivery. Currently, even the university's own mail system does not deliver to dorm rooms. Packages are delivered to a central location (lockers at Frist Campus Center) where students must go to retrieve them. This reinforces the fact that no logistics system on campus, whether internal or external, currently bridges the gap to the actual dorm room door.

3. Approach

To solve the "Last 500 Feet" problem, I moved away from the standard commercial delivery model. Instead of relying on outside drivers, I designed a system that works entirely within the campus walls. My approach is based on a hyperlocal logistics network. This means that every part of the system, from the customer and the driver to the delivery destination, is contained within the same small area, Princeton's campus. By keeping the network closed, UniEats can solve the security and access issues that stop companies like Uber and DoorDash.

3.1. The Student Run Model

The core of the UniEats approach is simple: designed for students, run by students. In this model, the workforce is not made up of full-time drivers. Instead, the drivers are fellow students. This serves two purposes. First, students have the necessary prox access to enter dorms and other campus buildings. Second, students already know the campus layout. An external driver might get lost looking for 1967 Hall, but a student will likely know exactly where it is and which door to use. This approach effectively removes the barriers that stop external deliveries. The security gate is no longer an obstacle because the driver is authorized to pass through it. The locked dorm door is no longer an obstacle because the driver has a key.

3.2. Role Management and Availability

Unlike commercial platforms that require completely separate apps for customers and drivers, UniEats manages these permissions within a single system. However, not every student is automatically a driver. By default, every new user who signs up is assigned the "Student" role. This allows them to place orders immediately. To become a driver, a user must be manually verified and assigned the "Driver" role via the Admin Dashboard. This ensures that only trusted users are handling food deliveries. Once a user has the "Driver" role, they can toggle their status. A driver is not forced to work set shifts. Instead, they use a simple "Go Online / Go Offline" toggle in the

app.

- **Offline:** When a driver is offline, they function just like a regular student. They can place orders for themselves.
- **Online:** When they toggle to online, they enter the driver workflow and begin receiving orders on a new driver dashboard.

This flexibility allows a student driver to earn money when they are free, but easily switch back to being a customer when they need to study or eat.

3.3. System Workflow

To make this network work effectively, the system has to manage orders in real time. The approach follows a specific flow, which guides the order from the moment it is placed until it reaches the dorm room.

Step 1: Order Creation

The process begins when a student places an order. In the pilot approach, UniEats focused on "Late Meal" deliveries. The student submits their order details, including their specific building and room number (e.g., Butler College, 1967 Hall, Room 123). Once the order is submitted and paid for, it enters a "Pending" state.

Step 2: Broadcasting

Once an order is created, the system needs to find a driver. Unlike Uber Eats, which assigns a specific driver to a rider, UniEats uses a more open approach. The system broadcasts the new order to all drivers who are currently marked as "Online." This helps address the fact that there will generally be fewer active drivers on UniEats than on more well-known food delivery services.

Step 3: The Claiming Process

Drivers see a list of available orders. They can review the pickup location and the drop-off dorm to decide whether to take the job. When a driver clicks "Claim Order," the system performs a crucial check. It locks the order in the database so that two drivers cannot claim the same meal at the same time. Once locked, the order disappears from the public feed and is assigned solely to that driver.

Step 4: Fulfillment and Delivery

Once the driver has claimed the order, they follow a simple process:

- **Pickup:** The driver goes to the pickup location (Frist Campus Center).
- **Transit:** The driver walks to the student's dorm. Because they are students, they can simply swipe into the building.
- **Delivery:** The driver goes to the specific room number. They knock on the door, hand over the food, and mark the order as "Delivered" in the app.

This workflow ensures that the "Last 500 Feet" are covered by the person best equipped to handle them: a fellow student with campus access.

4. Implementation

The UniEats system was designed as a full-stack web application, prioritizing responsiveness and mobile accessibility. Since the primary users are students who are often on the go, the application functions like a native mobile app. Below is an in-depth description of the frontend and backend design choices made during development.

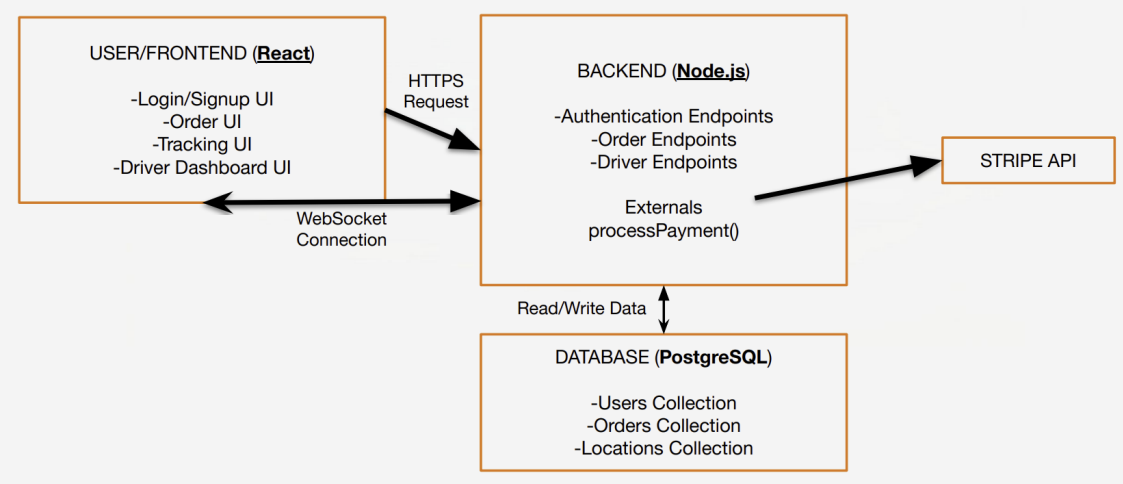


Figure 1: UniEats Architecture

4.1. Tech Stack and Architecture

Frontend: React

The frontend was made using React, a library chosen for its efficient handling of state updates and interactive view layers. React was chosen because it allows for a single-page application experience. This means that when a user navigates from the "Order" screen to the "Tracking" screen, the page does not need to reload in its entirety. This makes the app feel fast and responsive, which is critical for users on mobile data connections walking across campus.

Backend: Node.js and Express

The server logic is powered by Node.js running the Express framework. Node.js is particularly

useful for this project because of its event-driven nature, since a delivery app spends a lot of time waiting for specific events (waiting for a driver to claim an order, waiting for a payment to process, etc.). Node handles these multiple idle connections efficiently without slowing down the rest of the system.

Real Time: Socket.IO

One of the most important technical decisions was how to keep the student and driver in sync. In a traditional web app, the student's phone would have to ask the server every few seconds, "Has my food been picked up yet?" (also known as polling). This is inefficient and drains battery life. Instead, UniEats uses Socket.IO. This technology creates a permanent, open connection between the user's phone and the server. When a driver clicks "Picked Up," the server pushes the update to the student's device instantly. This enables the live, real-time tracking experience users expect from modern delivery apps.

Database: PostgreSQL

For storing data, the system uses PostgreSQL. Unlike simpler document stores, Postgres is a relational database. This is essential for UniEats because the data is highly structured and interconnected: a User places an Order, which a Driver claims. Postgres ensures that these relationships remain consistent and accurate.

Authentication: Azure Active Directory

To ensure the platform remains exclusive to the campus community, UniEats uses Passport.js with the Azure Active Directory (OpenID Connect, or OIDC) strategy. This integrates directly with the university's official login system. By using this standard, UniEats guarantees that every user has a valid @princeton.edu email address and a verified university identity (Azure OID), preventing unauthorized external users from joining the network.

4.2. Database Design and Schema

To ensure data integrity and manage relationships among users, orders, and campus locations, the system uses PostgreSQL. The database schema is designed around three main groups: Users, Orders, and Locations.

The Users Table

The users table acts as the central identity store. Importantly, it stores both customer and driver attributes in a single record, enabling the role-switching described in Section 3.

- **Identity Fields:** azure_oid (the unique ID from Princeton's login system) and email support secure single sign-on (SSO) authentication.
- **Role Management:** The role column ('student', 'driver', 'admin') determines permission levels.
- **Driver State:** The is_available boolean column allows drivers to toggle their visibility in the logistics pool without deleting their account or changing their role.

The Orders Table

The orders table is the transactional core of the application. It links a consumer (customer_id) to a provider (driver_id) and tracks the delivery lifecycle.

- **State Tracking:** The status column acts as a state machine, progressing through:

  $$\text{'pending'} \rightarrow \text{'claimed'} \rightarrow \text{'picked_up'} \rightarrow \text{'en_route'} \rightarrow \text{'delivered'}$$

- **Financial Data:** Fields like tip_amount and stripe_payment_id are stored for financial auditing and support resolution.
- **Ensuring a Single Driver:** The driver_id field is initially NULL. When a driver claims an order, this field is populated, locking the order to that specific user.

The Location Hierarchy

One of the unique challenges of a university campus is the complexity of addresses. A street address is insufficient. To solve this, the database uses a normalized hierarchy of three tables:

- **location_types:** Categories like "Residential College" or "Upperclassmen Hall".
- **locations:** The specific building names (e.g., "Butler College", "Yeh College").
- **halls:** The particular place within a building (e.g., "1967 Hall", "Bloomberg Hall").

This allows the frontend to offer cascading dropdown menus (Select College $\rightarrow$ Select Hall), ensuring that students cannot enter invalid or typo-filled addresses.

4.3. The Backend

Handling Concurrency

One of the most critical challenges in a food delivery application is the race to claim and order first. If two drivers see the same available order on their screens and press "Claim" at the exact millisecond, the system must ensure that only one driver is successful. To solve this, UniEats utilizes atomic database transactions rather than application-level checks. The claim endpoint executes a specific SQL update:

```
UPDATE orders
SET driver_id = $1, status = 'claimed'
WHERE id = $2 AND status = 'pending';
```

Including the condition `status = 'pending'` directly in the update query ensures atomicity. If Driver A's request hits the database first, the row is updated, and the status changes to 'claimed'. When Driver B's request arrives a millisecond later, the WHERE clause fails (because the status is no longer 'pending'), and the database returns zero modified rows. The API then catches this failure and returns a Conflict error, informing Driver B that the order is no longer available.

Real Time Communication with Socket.IO

To efficiently route messages, the backend uses Socket.IO's "Rooms" feature. Rather than broadcasting every message to every connected user, the system segments connections based on identity and role:

- **user:{id} Room:** Upon login, every user joins a private room keyed to their unique User ID. This allows the server to send private updates ("Your order was picked up") only to the specific customer involved.
- **role:driver Room:** Users with the driver role join this shared room. When a new order is created, the server emits the order_created event specifically to this room, ensuring that only active drivers receive the alert for new work.

State Management and Driver Availability

The system maintains a strict state machine to track the physical progress of orders. Drivers use a series of endpoints to transition an order through the picked_up, en_route, and delivered states. Additionally, the system manages supply through the driver availability endpoint. This allows student drivers to toggle their is_available status. The order creation endpoint checks this pool before accepting a payment; if the query: `SELECT COUNT(*) FROM users WHERE role='driver' AND is_available=true` returns zero, the server rejects the new order with a Service Unavailable error. This prevents students from paying for delivery when no one is online to deliver it.

Payment and Promo Logic

Financial logic is handled on the server to prevent tampering. The create-payment-intent endpoint calculates the total price (Service Fee + Tip). It also handles the logic for the "WelcomeBite" promotion, which allows new users to waive the service fee on their first order. The server verifies the promo code string and then queries the database to check whether the user has any prior orders with `COUNT(*) = 0`. Only if both conditions are met does the server waive the base service

fee. This ensures that a user cannot bypass validation by simply manipulating frontend JavaScript variables.

4.4. The Frontend: Student Dashboard

The Student Interface is the most essential part of the application because it is what the customer actually sees and uses. It was built using React to ensure it feels fast and responsive, like a native mobile app. The interface focuses on two main tasks: placing an order and tracking it in real time.

The Ordering Workflow (Integration with Frist)

One of the biggest challenges in this implementation was that UniEats lacked direct access to the university's dining menu and ordering system. To solve this, I designed a simple workflow in the NewOrder.js component. Instead of selecting food items directly in UniEats, the app guides the student to use the existing Frist Campus Center ordering website.

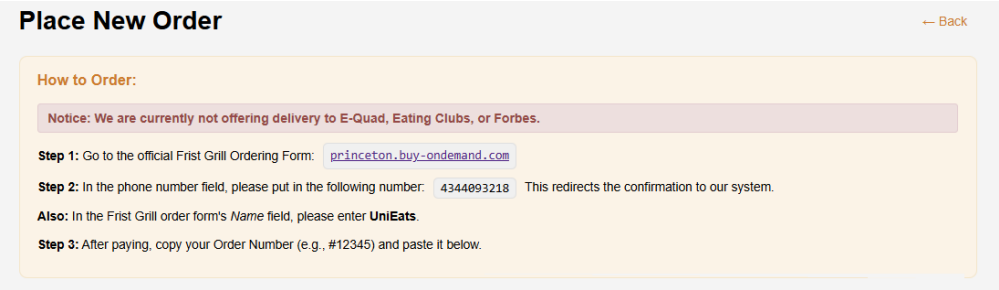


Figure 2: Delivery Order Instructions

As seen in Figure 2, the app displays an instruction box that tells the student to place their order on princeton.buy-ondemand.com. They then enter the specific UniEats support phone number into the contact field (so text updates go to the system, not the student). Then they copy their Order Number and return to the UniEats app. The student then enters this Order Number into the UniEats delivery form. This allows the system to track delivery logistics without managing the kitchen's complex menu data.

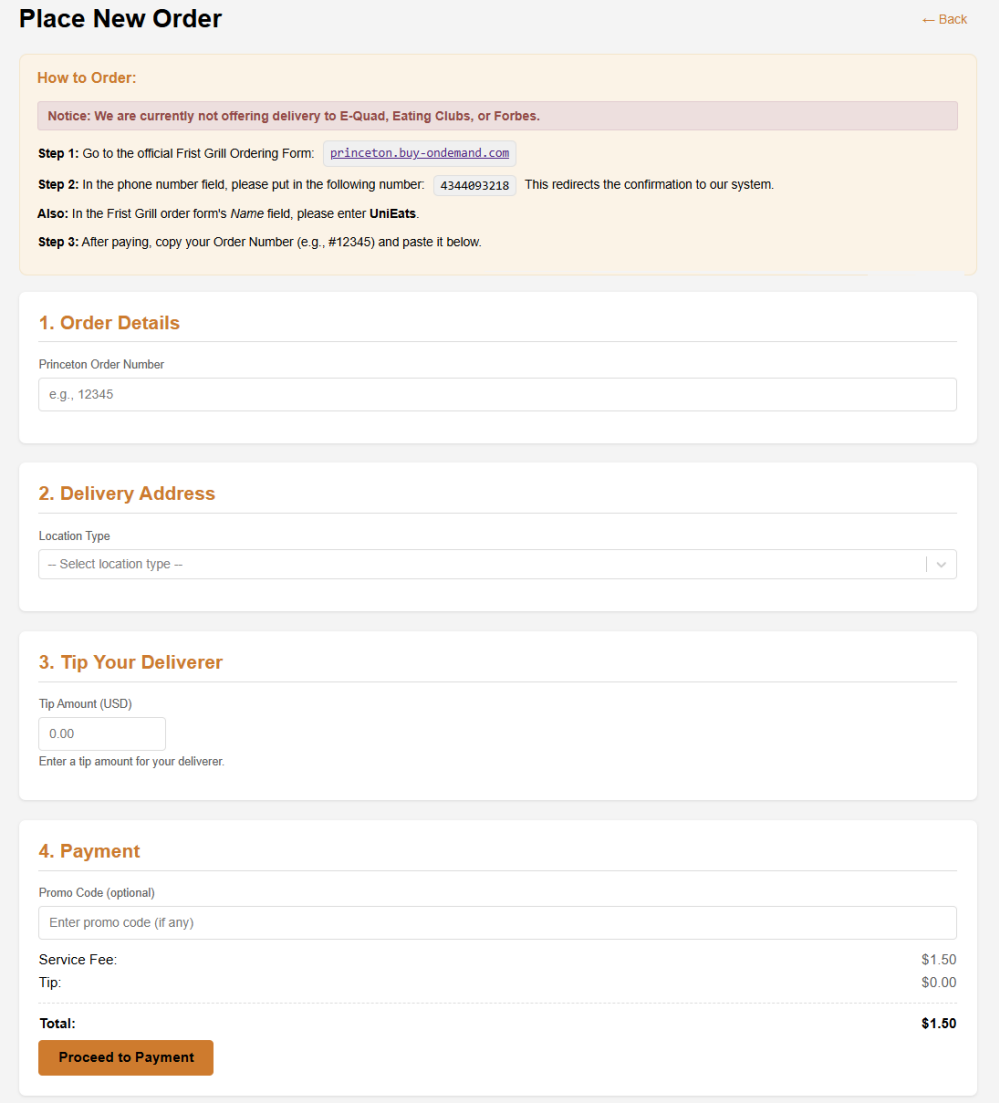


Figure 3: Full Delivery Order Form

In Figure 3, we can see the complete delivery order form, with fields for the order number, delivery address, tip, and promo code/payment method. Once a user presses the Proceed to Payment button, they are met with the Stripe interface to complete their transaction.

Handling Campus Addresses To prevent students from entering vague addresses like "My dorm," the ordering form uses a dynamic selection system, briefly covered in the database section.

- **Location Type:** The user first selects whether they are in a "Residential College," an "Up-

perclassmen Hall," or a "Campus Building".

- **Dynamic Dropdowns:** If they select, for instance, "Residential College," the app automatically fetches the list of colleges (e.g., Butler, Yeh) from the API. Once a college is chosen, a second dropdown appears listing only the specific halls within that college.

This logic ensures that a student cannot accidentally select a non-existent hall, reducing confusion for the driver later.

The Live Tracker (Visualizing the Delivery) Once the order is placed and paid for via the Stripe modal, the student is moved to the dashboard. This screen (StudentDashboard.js) features a Live Tracker that shows the delivery's progress. The tracker visualizes the five stages of the delivery lifecycle, shown in Figure 4 below.

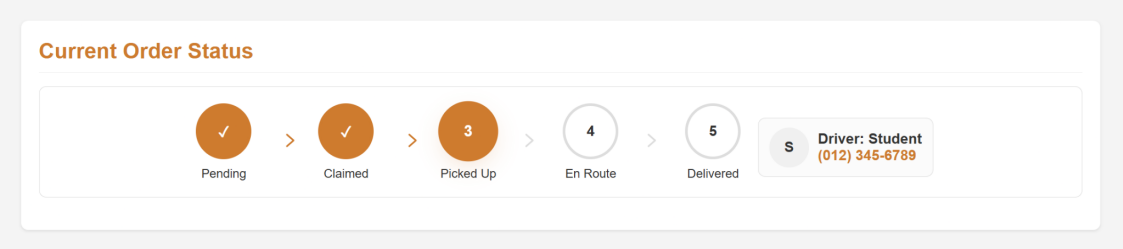


Figure 4: Live Order Tracker on Desktop

Behind the scenes, this component listens for real-time updates using the socket.on ('order_updated') event. Whenever the driver presses a button on their screen, the student's screen updates instantly.

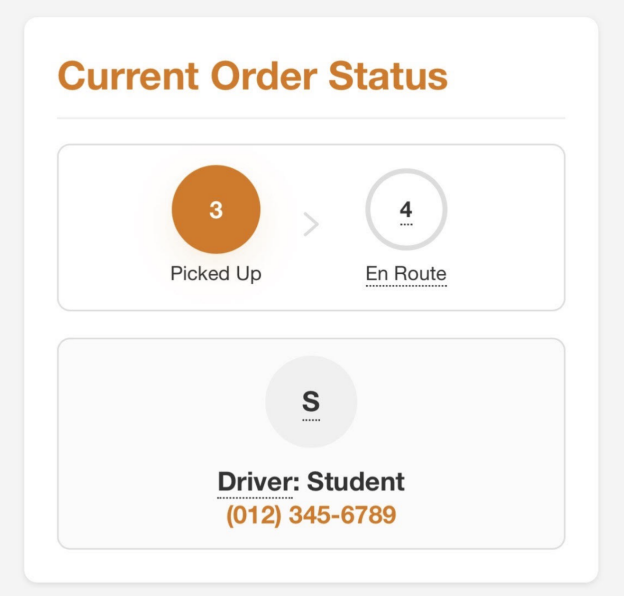


Figure 5: Live Order Tracker on Mobile

The code includes a check for isSmallScreen (window width less than 600px). If the user is on a phone, the tracker adjusts its layout to fit the smaller screen, as seen in the difference between Figures 4 and 5, ensuring the status steps are still readable. Once a driver claims the order, their name and contact button appear below or next to the tracker, allowing the student to call them if there is an issue.

4.5. The Frontend: Driver Dashboard

The Driver Dashboard is designed for utility and speed. While the student interface focuses on aesthetics and tracking, the driver interface focuses on task management. It acts as a command center where students can switch into work mode, find orders, and manage their delivery queue.

Availability Management ("Go Online")

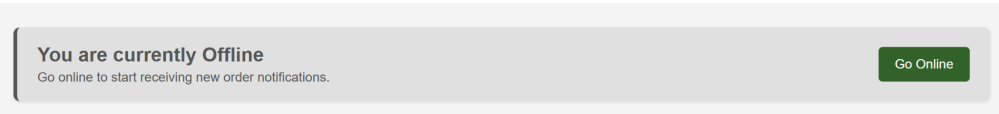


Figure 6: Driver Dashboard Offline Banner

The first interaction for any driver is the status toggle. By default, a driver is "Offline," meaning they do not receive notifications and cannot see the order feed. To start a shift, the driver clicks the "Go Online" button, as seen in Figure 6. This triggers the handleAvailabilityToggle function, which sends a PUT request to availability and puts the driver into "Online" mode.

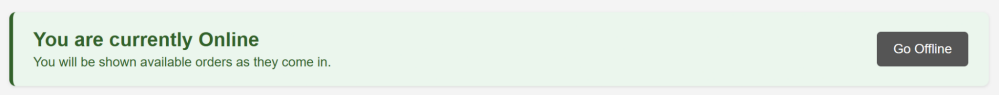


Figure 7: Driver Dashboard Online Banner

The interface immediately updates to display a green "You are currently Online" banner, as shown in Figure 7. This action flips the is_available boolean in the database, adding the driver to the active pool so the system allows new orders to be placed.

The Dual Feed

To organize tasks, the dashboard is split into two distinct feeds:

- **Available Orders (The Public Pool):** This list displays all pending orders that have been broadcast to the network. Every online driver sees this same list. It shows essential information such as order number, delivery location, and tip amount. This allows the driver to make an informed decision before claiming an order.
- **My Claimed Orders (The Private Queue):** Once a driver claims an order, it moves to this private list. Crucially, sensitive contact information (such as the student's phone number) is revealed only after the order is transferred to this view.

The Delivery Workflow

The dashboard guides the driver through the physical steps of delivery using dynamic action buttons. The interface checks the status of each order and renders the appropriate next step:

- Status: claimed $\rightarrow$ Button: "Mark as Picked Up"

- Status: picked_up → Button: “Mark as On the Way” or “Mark as Delivered”
- Status: en_route → Button: “Mark as Delivered”
- Status: delivered → Order is removed from Driver Dashboard

When a button is clicked, the app calls updateOrderStatus, which updates the database and immediately pushes a WebSocket event to the specific student’s phone. This ensures the driver never has to text the student updates manually; the app handles communication automatically.

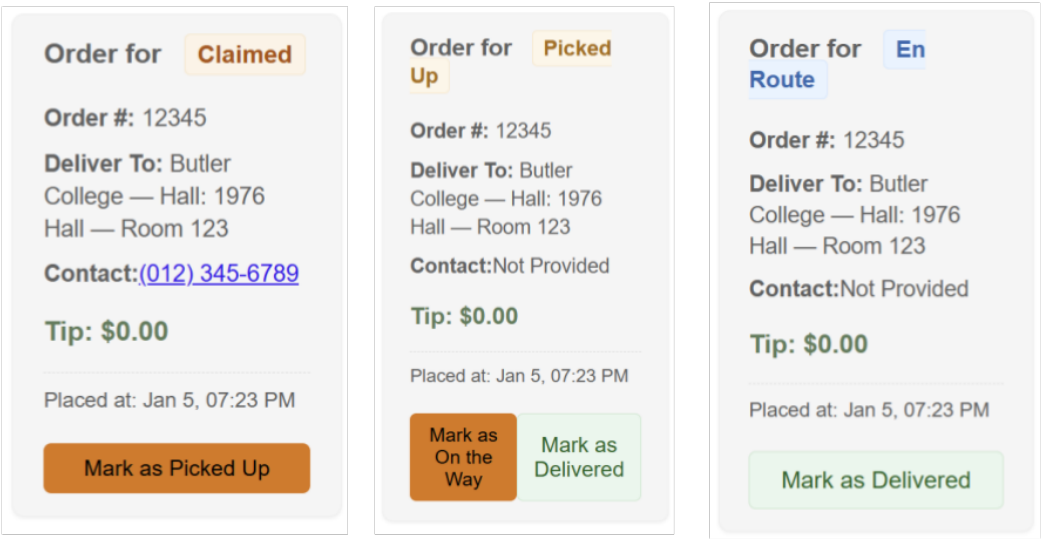


Figure 8: Possible Order Cards Depending on Delivery Status

Real Time Synchronization

To prevent frustration, the “Available Orders” feed must be perfectly synced across all drivers. If Driver A claims an order, it must disappear from Driver B’s screen instantly. The dashboard achieves this by listening for the order_claimed socket event. When this event fires, the app filters the local state array (setAvailableOrders) to remove the claimed ID immediately, preventing double-claiming errors where two drivers try to claim the same job.

4.6. The Admin Center

While the Student and Driver interfaces are designed for limited, role-specific tasks, the Admin Center provides a view of the entire system. This dashboard was built to give me complete control over the application's data without running manual SQL queries against the database. It served as both a management console for the pilot and a critical debugging tool during testing.

User Role Management The most essential function of the Admin Center is Role Assignment. As described in Section 3, all new users start as "Students" by default. To build the driver workforce, I needed a way to promote specific trusted users manually. The Admin interface fetches a list of all registered users with GET /api/admin/users. It displays them in a table, with a button next to each person's current role. When I select "Set as Driver," the app sends a PUT request to update that user's permissions. This is how I set up the initial cohort of drivers for the November pilot.

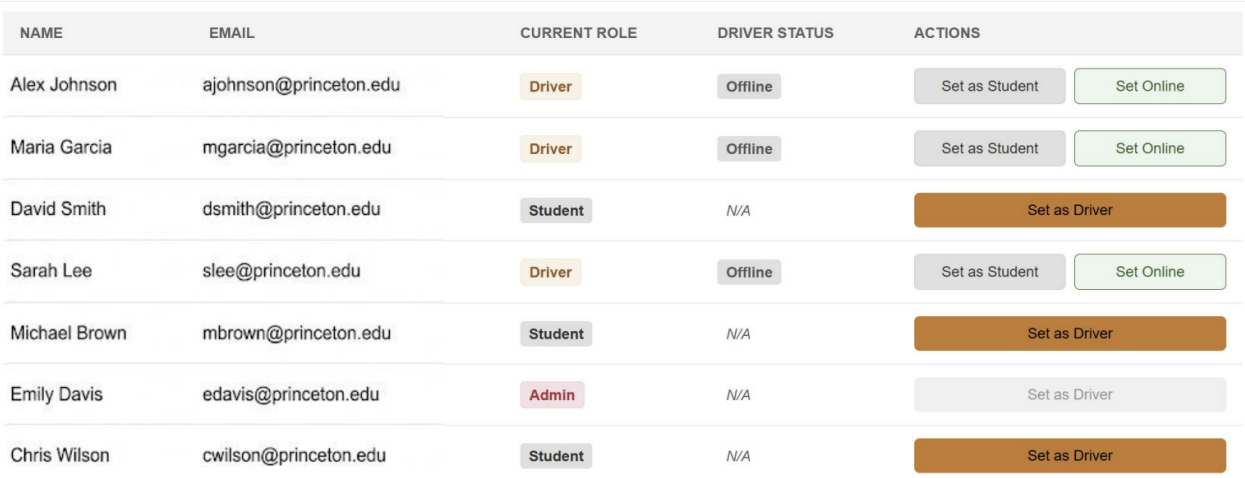


Figure 9: User Role Management in Admin Center (Users Anonymized)

Order Management

The Admin Center also provides a global feed of every order in the system, regardless of status. This view includes details that regular drivers cannot see, such as the full customer name and email. During the pilot, this feature was essential for monitoring the system's health. I could watch orders

come in in real time and ensure they were being claimed. If a test order was placed by mistake, or if an order got stuck due to a bug, I included a Delete function to remove the bad data and keep the driver feed clean.

Testing and Simulation

Beyond management, the Admin Center was the primary tool used for testing the application. Because the "Student" and "Driver" views are entirely separate, it is difficult to test the full delivery flow with just one account. As the Admin, I could easily access all role functions dynamically. I would:

- Act as a Student to place a test order.
- Use the Driver view to see if orders appeared in the feed.
- Claim orders and mark them as delivered.
- Back at the Admin view, I could verify the database records.

This ability to act in all roles and force update driver availability allowed me to simulate complex scenarios, like what happens when no drivers are online, without needing multiple physical devices or test users.

5. Evaluation

To validate the system, model, and technical stability of the UniEats platform, a pilot study was conducted on the Princeton University campus. The primary goal of this evaluation was to answer three main questions:

- **Technical Viability:** Can the system successfully perform a delivery from creation to completion without any major hassle?
- **Operational Viability:** Can student drivers successfully navigate the "Last 500 Feet" to find specific dorm rooms using their own prox access?
- **Commercial Viability:** Will students on campus use the platform?

5.1. Pilot Methodology

Duration and Scope

The pilot was conducted over four days from November 17 to November 20, 2025. The scope was intentionally limited to test the core logistics before scaling. The pilot focused exclusively on "Late Meal" orders from the Frist Campus Center. This location was chosen because it is a high-traffic hub for student dining. The service operated only during Late Meal hours (8:30 PM - 10:00 PM), matching peak demand. A total of 40 students expressed initial interest via a sign-up form distributed before the launch. All of them were let into the pilot program. Lastly, a small team of 3 drivers was chosen to help fulfill orders, with each driver picking different nights to work a shift.

The Frist Bridge Workflow

A significant constraint of this pilot was the lack of an official API integration with Campus Dining. UniEats could not digitally inject orders into the Frist kitchen system. To overcome this, the pilot utilized the Frist Bridge workflow described in Section 4.4. Participants were instructed to place their food orders through the official Frist dining portal (princeton.buy-ondemand.com), but

were asked to input a specific UniEats Support Number into the contact field. This ensured that when the kitchen marked an order as Ready, the text notification was sent to my team, rather than the student's personal device. The student then manually inputs their confirmation number into the UniEats app to initiate the delivery request.

Privacy and Safety

To ensure safety during the pilot, driver accounts were restricted to known students who were manually verified via the Admin Console. Furthermore, delivery locations were limited to primary campus residential colleges; distant places like the E-Quad or off-campus Eating Clubs were excluded from the service zone to keep transit times manageable.

5.2. Quantitative Results

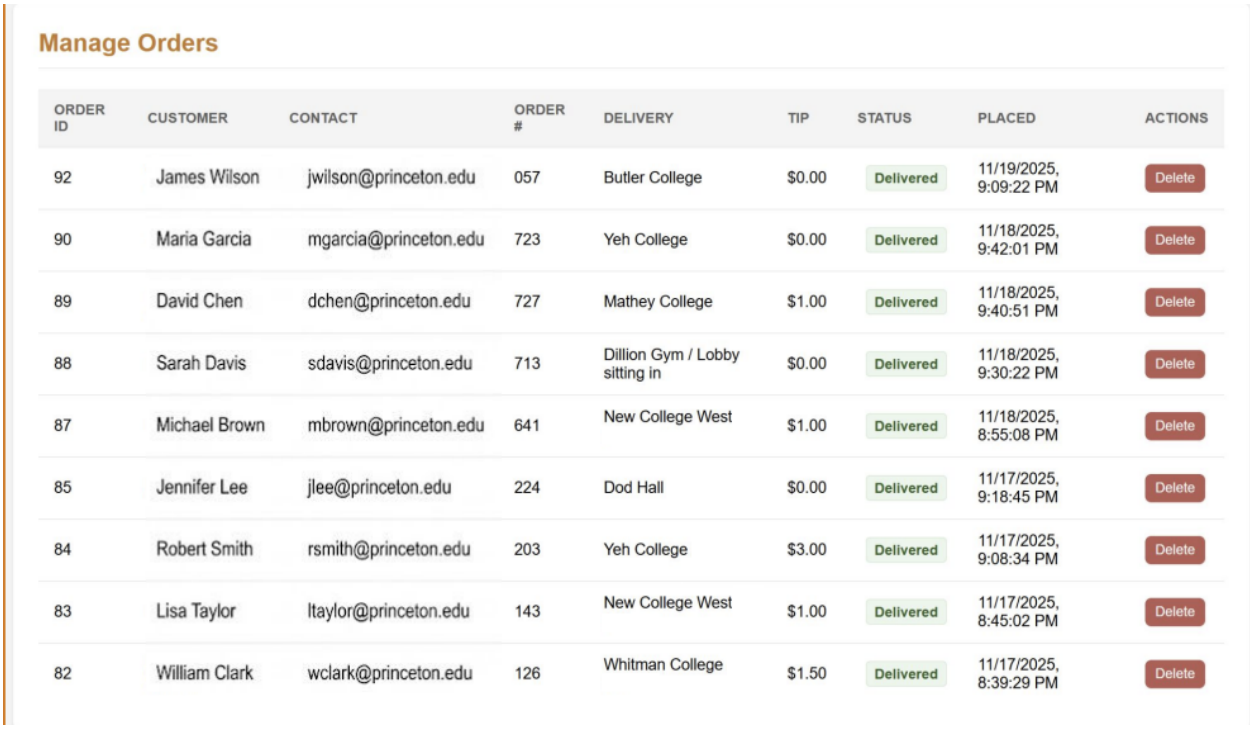
The quantitative data collected during the pilot demonstrate the system's operational success.

Delivery Success Rate

The most critical metric for the pilot was the completion rate.

- Total Orders Placed: 9
- Total Orders Delivered: 9
- Success Rate: 100%

Every order placed in the system was successfully claimed by a driver, picked up from Frist, and delivered to the correct dorm room. There were no cancellations, no undeliverable errors, and no instances where a student had to leave their room to meet a driver. This 100% success rate stands in stark contrast to the frequent failures of external delivery services on campus.



Manage Orders

ORDER ID	CUSTOMER	CONTACT	ORDER #	DELIVERY	TIP	STATUS	PLACED	ACTIONS
92	James Wilson	jwilson@princeton.edu	057	Butler College	$0.00	Delivered	11/19/2025, 9:09:22 PM	Delete
90	Maria Garcia	mgarcia@princeton.edu	723	Yeh College	$0.00	Delivered	11/18/2025, 9:42:01 PM	Delete
89	David Chen	dchen@princeton.edu	727	Mathey College	$1.00	Delivered	11/18/2025, 9:40:51 PM	Delete
88	Sarah Davis	sdavis@princeton.edu	713	Dillion Gym / Lobby sitting in	$0.00	Delivered	11/18/2025, 9:30:22 PM	Delete
87	Michael Brown	mbrown@princeton.edu	641	New College West	$1.00	Delivered	11/18/2025, 8:55:08 PM	Delete
85	Jennifer Lee	jlee@princeton.edu	224	Dod Hall	$0.00	Delivered	11/17/2025, 9:18:45 PM	Delete
84	Robert Smith	rsmith@princeton.edu	203	Yeh College	$3.00	Delivered	11/17/2025, 9:08:34 PM	Delete
83	Lisa Taylor	ltaylor@princeton.edu	143	New College West	$1.00	Delivered	11/17/2025, 8:45:02 PM	Delete
82	William Clark	wclark@princeton.edu	126	Whitman College	$1.50	Delivered	11/17/2025, 8:39:29 PM	Delete

Figure 10: Admin View of Orders During Pilot (Users Anonymized)

Geographic Distribution and Access

The pilot successfully tested the prox access capabilities of the drivers across a diverse set of campus locations. As shown in Figure 10, deliveries were made to multiple distinct residential colleges, including Butler, Yeh, Mathey, New College West, Whitman, Dod, and even Dillion Gym. This wide distribution confirms that the student-driver model is not limited to a single area. Drivers successfully navigated and entered buildings in at least five different residential areas, demonstrating that the student workforce has the necessary access and knowledge to cover the majority of the campus.

Financial Data and Tips

The system also successfully processed real financial transactions.

- **Service Fees:** A flat fee of $1.50 per order was charged (though waived for most via the "WelcomeBite" promo).

- **Tipping Behavior:** Students were willing to tip for convenience. For orders with non-zero tips, the tips ranged from $1.00 to $3.00 per order, with multiple orders including tips of $1.00 or $1.50.

5.3. Qualitative Results

While quantitative data demonstrates the feasibility of the logistics, qualitative feedback highlights the friction in the current prototype and the strong market demand for a permanent solution.

User Experience and Friction Points

The primary source of user friction during the pilot was the "Frist Bridge" ordering workflow. As detailed in the implementation, students were required to perform a manual application swap. The specific instruction to "Go to the official Frist Grill Ordering Form... enter the UniEats phone number... copy your Order Number... and paste it below" proved to be a significant barrier to entry. Feedback indicated that this multi-step process was inconvenient for some users. While 40 students expressed interest, only 9 completed the ordering flow (despite having 21 users make accounts), suggesting that a seamless, direct integration with Campus Dining APIs is necessary to capture the full market potential.

Market Validation and Student Demand

Despite the friction, the pilot received strong external validation. The service was featured in The Daily Princetonian in an article titled "Student-run initiative pilots Late Meal delivery". Interviews conducted by the campus newspaper confirmed the hypothesis behind the entire project. Students explicitly stated that the lack of delivery prevents them from utilizing campus dining options. One student noted, "I'd probably order late meal more often on nights when I'm busy or don't feel like walking across campus". Another stated, "I know so many people who always talk about wishing they could have their late meal delivered to them". This confirms that the "Last 500 Feet"

is a recognized pain point among the student body and that the 100% success rate of the pilot has effectively proven a viable solution.

6. Conclusions and Future Work

The “Last 500 Feet” of delivery logistics has long been an unsolved problem for closed university campuses. Commercial giants like Uber Eats and DoorDash have successfully conquered the “Last Mile.” Still, their model breaks down at the campus gate due to security restrictions and a lack of vehicle access. This project, UniEats, proposes a solution: a hyperlocal logistics network that utilizes the students themselves as the delivery workforce.

6.1. Summary of Contributions

Proof of Concept

UniEats demonstrated that a student-run network can effectively bypass campus security barriers. By leveraging the existing prox access for student drivers, it achieved a 100% delivery success rate during the pilot, proving that the physical obstacles posed by locked dorms can be overcome without compromising campus security.

Technical Stability

The full-stack implementation, built on React and Node.js with real-time WebSocket communication, proved stable under real-world conditions. The system successfully handled concurrent orders, Stripe financial transactions, and live state updates without failure.

Market Demand

Qualitative feedback and press coverage from The Daily Princetonian confirmed that the demand for dorm-to-door delivery is high. Students are willing to pay for convenience, specifically when academic workload or illness makes walking to dining halls difficult.

6.2. Limitations

Despite the pilot's success, the current system has limitations that must be addressed before a full-scale launch.

The Frist Bridge Friction

The lack of official API access to Campus Dining forced UniEats to use a manual workaround where students copied and pasted order numbers. This added friction and likely reduced the conversion rate of interested users. A direct integration or partnership with dining services is required for a seamless experience.

Limited Scope

The pilot was restricted to specific residential colleges and Late Meal hours. Distant locations like the E-Quad and Eating Clubs were excluded, leaving a portion of the student body unserved. Additionally, off-campus local dining options were unavailable due to time constraints.

6.3. Future Work

The UniEats roadmap extends through the next academic year, intending to evolve from a semester project into a permanent campus service.

Phase 2 Driver Onboarding

The immediate next step is to formalize the labor supply. UniEats needs to recruit and train an initial cohort of 10-15 dedicated drivers. This will ensure consistent availability and allow UniEats to expand operating hours beyond the pilot window.

Phase 3: Local Partnerships

To help address the Frist Bridge limitation, UniEats needs to expand its options by partnering with local off-campus vendors. Restaurants like Tacoria are popular with students but suffer from the same access issues as major delivery apps. By handling the "Last 500 Feet" for these vendors, UniEats can offer a better user experience without needing deeper integration into University Dining systems.

Phase 4: Full Launch

The long-term goal of UniEats is to migrate from a web-based application to a native mobile application deployed on the iOS App Store and Google Play Store. A native app will allow for better location tracking, push notifications for order updates, and a smoother payment experience.

References

[1] Curry, D. (2025, December 17). *Food delivery app revenue and usage statistics (2025)*. Business of Apps. https://www.businessofapps.com/data/food-delivery-app-market/

[2] Kim, J. (2025, December 1). *Student-run initiative pilots Late Meal delivery*. The Daily Princetonian. https://www.dailyprincetonian.com/article/2025/12/princeton-news-stlife-delivery-pilot-program-for-late-meal

Appendix A (Frontend Designs)

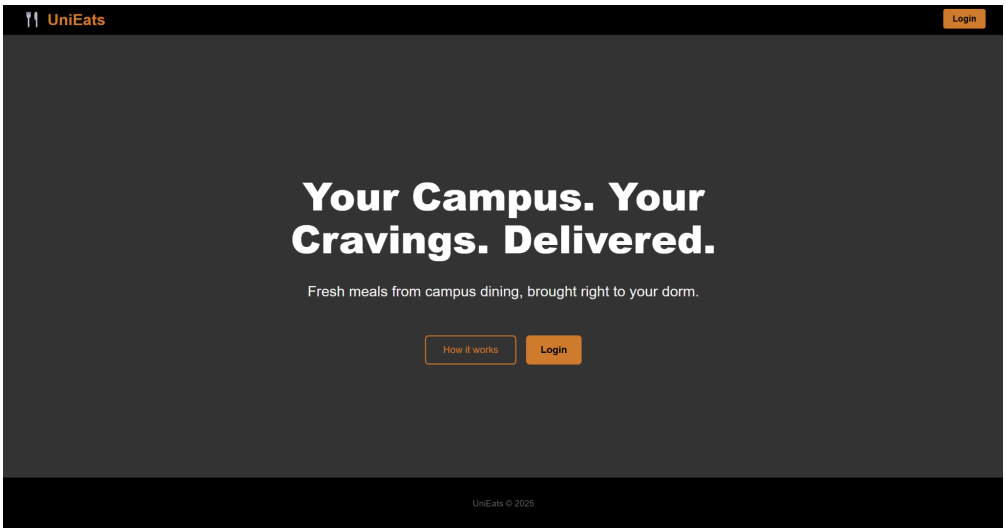


Figure A1: Landing Page

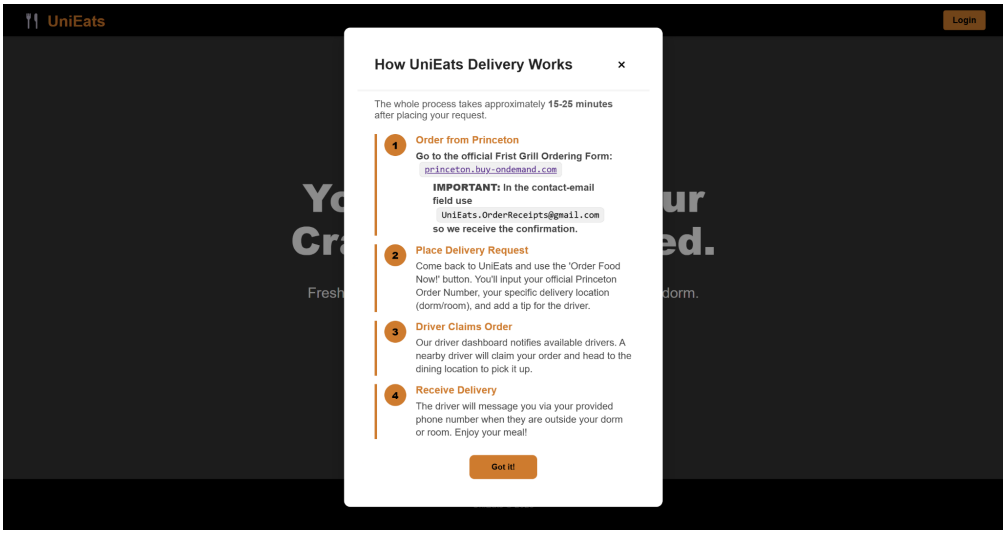


Figure A2: Pop-up after clicking “How It Works” in A1

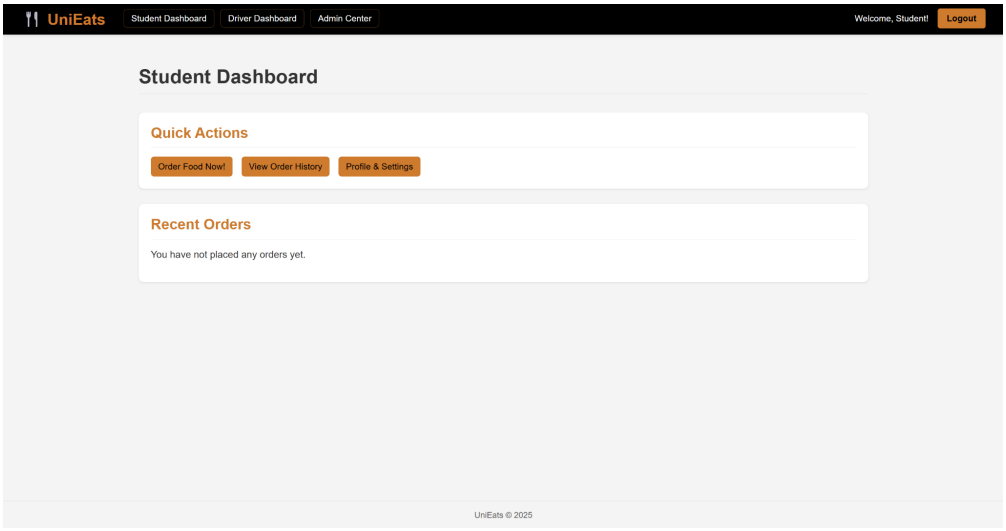


Figure A3: Student Dashboard

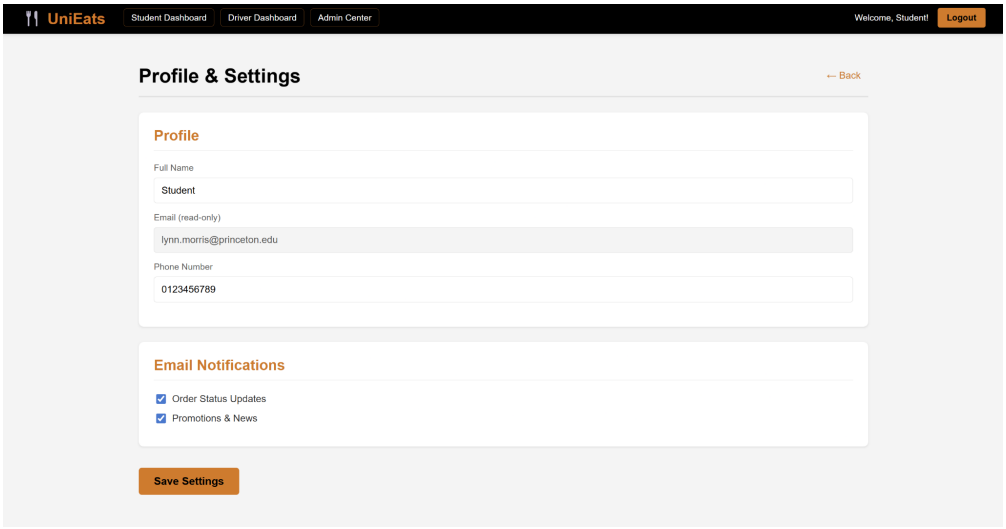


Figure A4: Profile and Settings

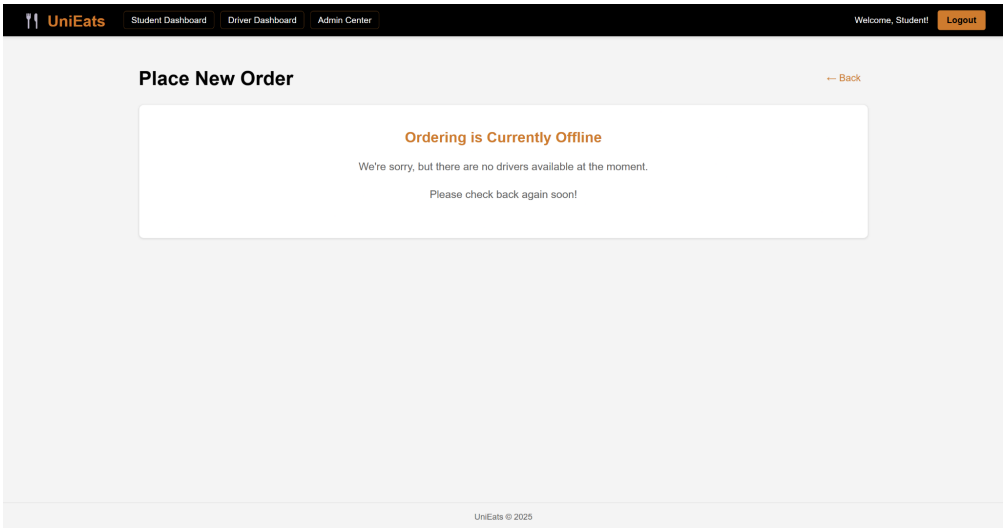


Figure A5: Order Screen When No Drivers Online

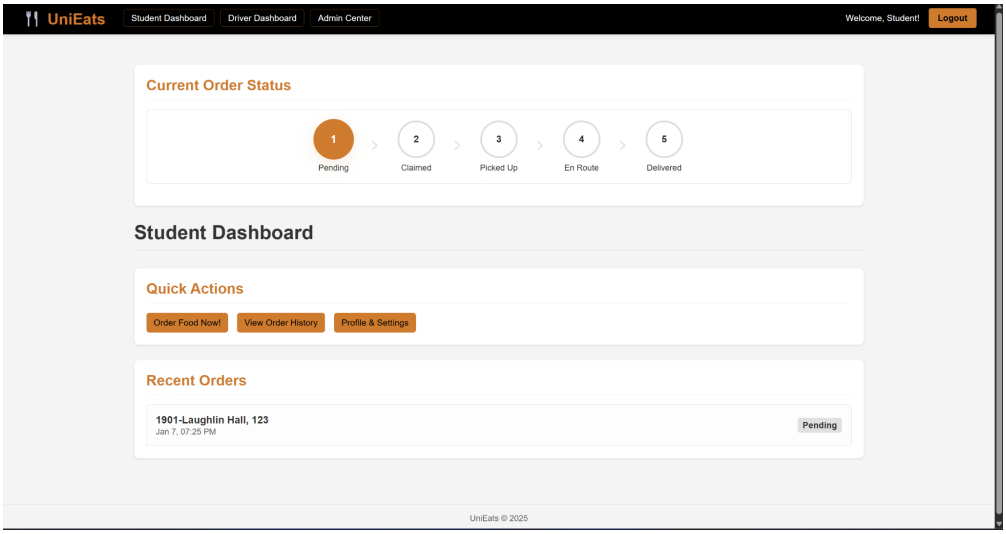


Figure A6: Student Dashboard When Having an Active Order

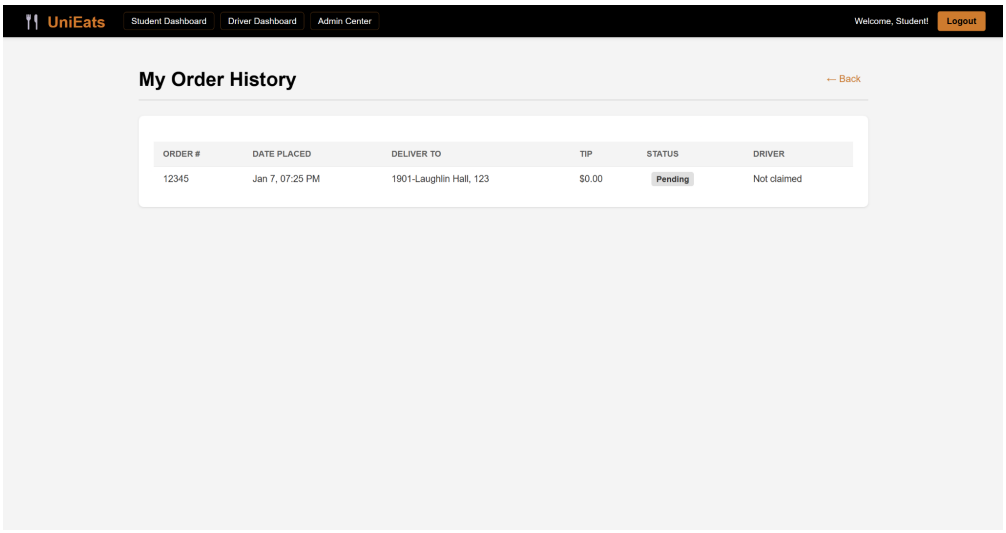


Figure A7: Order History

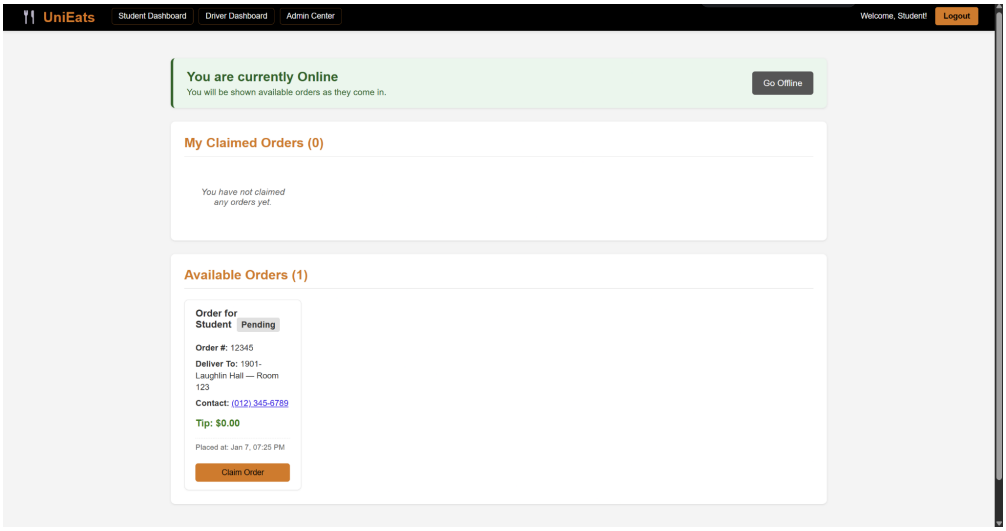


Figure A8: Driver Dashboard Toggled Online With Available Order

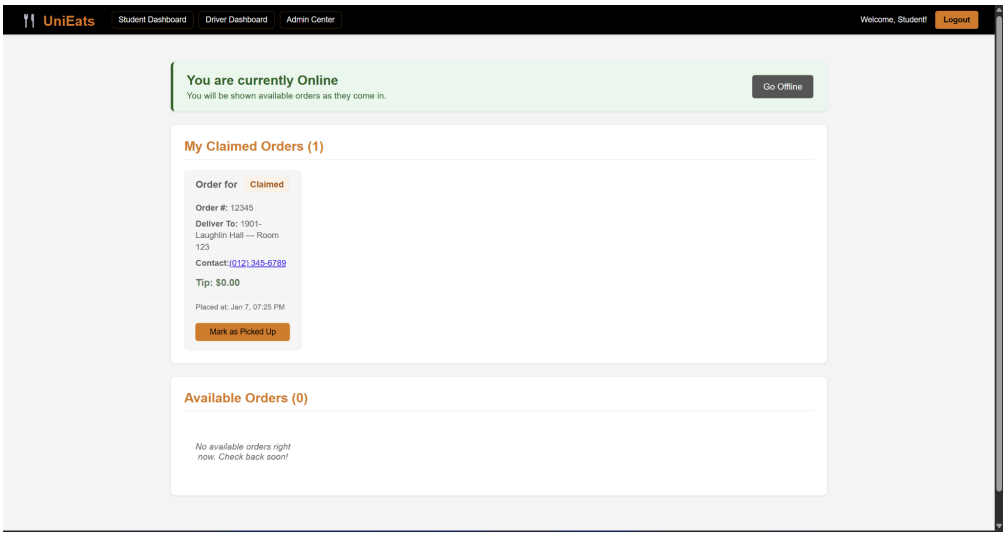


Figure A9: Driver Dashboard With Claimed Order